

F l y w e i g h t

Flyweight Pattern

Share to save memory

DEFINITION

Use sharing to support huge numbers of fine-grained objects by separating intrinsic (shared) from extrinsic (context) state.

Flyweight supports huge numbers of fine-grained objects by splitting state: intrinsic state (shared, context-free) lives in one shared object, and extrinsic state (per-use, contextual) is passed in by the client. A factory pools the shared instances and hands back the same one per key — trading CPU for RAM.

WHY IT MATTERS

Millions of similar objects — glyphs, tiles, particles — blow memory if each is a full standalone instance. Flyweight shares the common, immutable part across all of them and supplies the varying part per call, so thousands of references point to a handful of objects. The costs: the shared state must be immutable (one object backs many contexts), and the extrinsic state must be computed and passed everywhere.

— How it works

Flyweight Stores intrinsic state; its operations take extrinsic state as arguments.

FlyweightFactory Creates and pools flyweights, returning shared ones by key.

Client Holds the extrinsic state and passes it into flyweight operations.

HOW TO APPLY

1. Split the state into intrinsic (shareable) and extrinsic (per-context).
2. Make the intrinsic part immutable and move it into the flyweight.
3. Pool flyweights in a factory; return the same instance per key.
4. Pass extrinsic state into each operation; never let clients new flyweights.

LITMUS TEST

Are you about to create a huge number of objects whose state splits cleanly into shared vs contextual? If most of each object is identical, share it and pass the rest in.

— Smell → fix

One full object per glyph — millions of near-identical instances:

```
// one object per character on the page
chars.map(c => new Glyph(c, font, x, y))
// a 1M-char doc → 1M objects, mostly duplicate
```

↓ SHARE INTRINSIC, PASS EXTRINSIC IN

```
const pool = new Map() // the factory cache
function glyph(ch) {
  if (!pool.has(ch)) pool.set(ch, { ch }) // shared
  return pool.get(ch)
}
// extrinsic x, y, font passed at draw time:
draw(glyph('A'), x, y, font) // 'A' shared everywhere
```

glyph('A') returns the one shared 'A' object for the whole document; position and font are passed at draw time. A million characters now reference a few dozen flyweights. The shared part must stay immutable, and the factory is mandatory — clients must never new one.

USE WHEN

- ✓ Millions of similar objects (glyphs, tiles, particles)

AVOID WHEN

- ▲ Few objects, or state that can't be split cleanly

— Trade-offs & pitfalls

PROS

- + Big memory savings
- + Cache reuse

CONS

- − Added complexity
- − Extrinsic state must be passed in everywhere

COMMON PITFALLS

- ▲ Shared state must be immutable: a client mutating intrinsic data corrupts every context.
- ▲ The factory is mandatory — if clients new flyweights directly, sharing breaks.
- ▲ Only worth it at scale; watch the pool size — an unbounded pool is a leak (bound it or weakly reference).

WHERE YOU SEE IT

- ▶ Character glyphs in editors (one object per code, shared across the doc — GoF's example).
- ▶ Game sprites / particles: one mesh shared by thousands, position passed in (GPU instancing).
- ▶ Java Integer cache (−128..127) and JS / Java string interning.

SEE ALSO

Composite	flyweights are often a Composite's shared leaf nodes
Singleton	the flyweight factory is commonly kept as a single instance
Prototype	opposite move — Prototype copies instances, Flyweight shares one
State	stateless State or Strategy objects can be shared as flyweights

— Interview & sources

Intrinsic vs extrinsic state?

Intrinsic is shared and context-free (lives in the flyweight); extrinsic is per-use, passed in by the client.

Flyweight vs object pool?

Flyweight shares immutable objects concurrently; a pool lends out and recycles mutable ones one at a time.

Why must shared state be immutable?

Because one object backs many contexts — any mutation leaks into all of them.

Is string interning a flyweight?

Yes — equal literals share one immutable instance handed out by a pool.

REMEMBER

Share one immutable copy of the state objects have in common; pass the rest in from outside.

SOURCES

GoF — "Design Patterns" (1994): Flyweight (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Flyweight (leftover patterns)